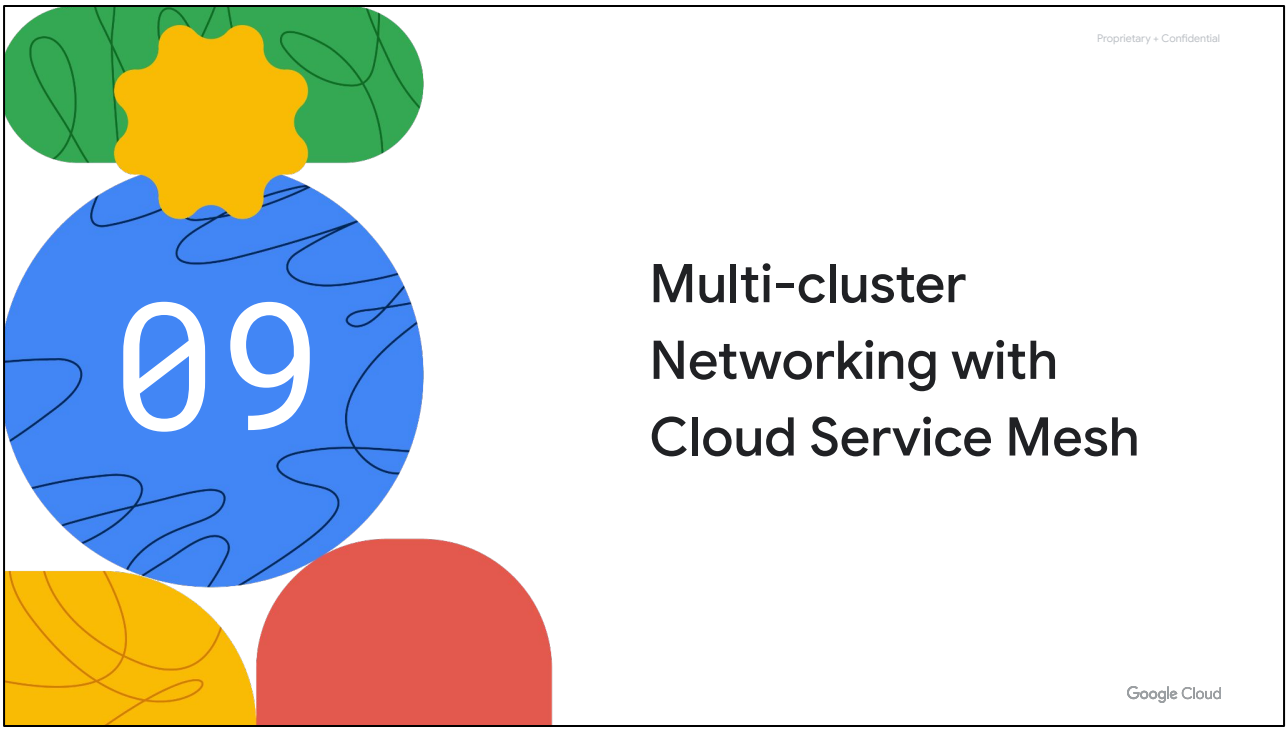


In multi-cluster mesh networking, there are different ways to configure a service mesh.

The main differences depend on the network and the type of cluster.

Proprietary + Confidential

Multi-cluster Networking with Cloud Service Mesh

01 Single network east-west routing

02 Multiple network east-west routing

Google Cloud

In this section, titled Multi-cluster Networking with Cloud Service Mesh, you learn to:

- Set up a multi-cluster mesh with a single subnet in a single VPC network and account for variations like multi-region clusters, multiple projects, shared VPC, and private clusters.
- Enable communication between GKE clusters on different networks using an east-west gateway and attached clusters.

Proprietary + Confidential

Multi-cluster Networking with Cloud Service Mesh

01 Single network east-west routing

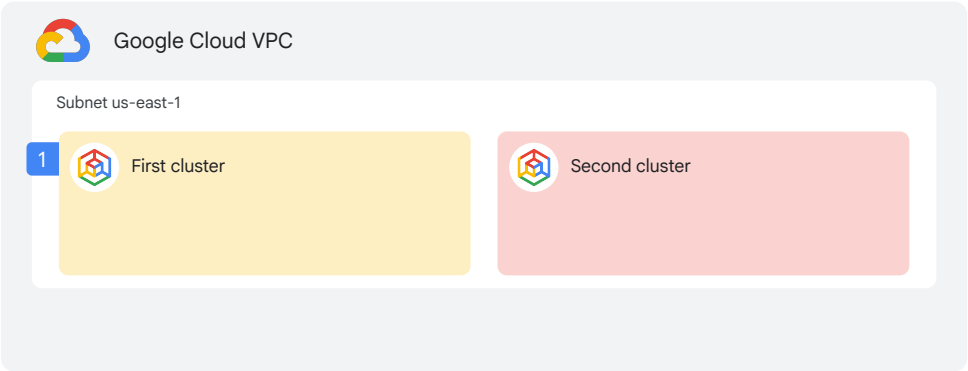
02 Multiple network east-west routing

Google Cloud

Let’s begin by demonstrating a multi-cluster mesh setup with a single subnet in a single VPC network.

Proprietary + Confidential

Create and register your GKE clusters to fleet

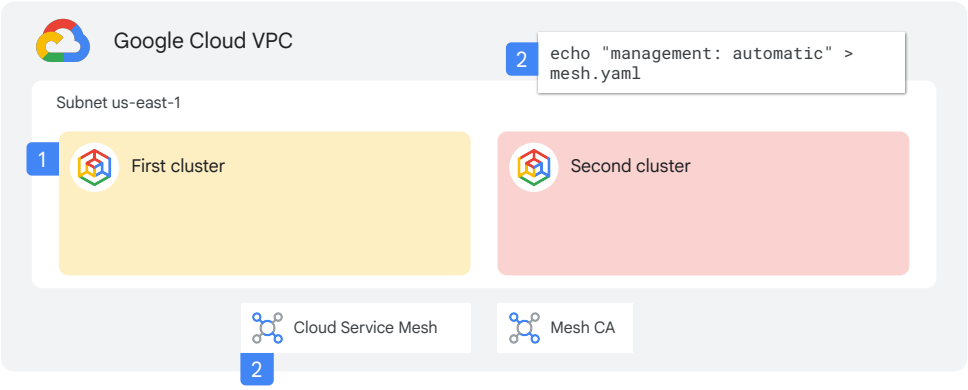


Google Cloud

Step one is to create and register your GKE clusters to a fleet.

Proprietary + Confidential

Install managed Cloud Service Mesh on both clusters



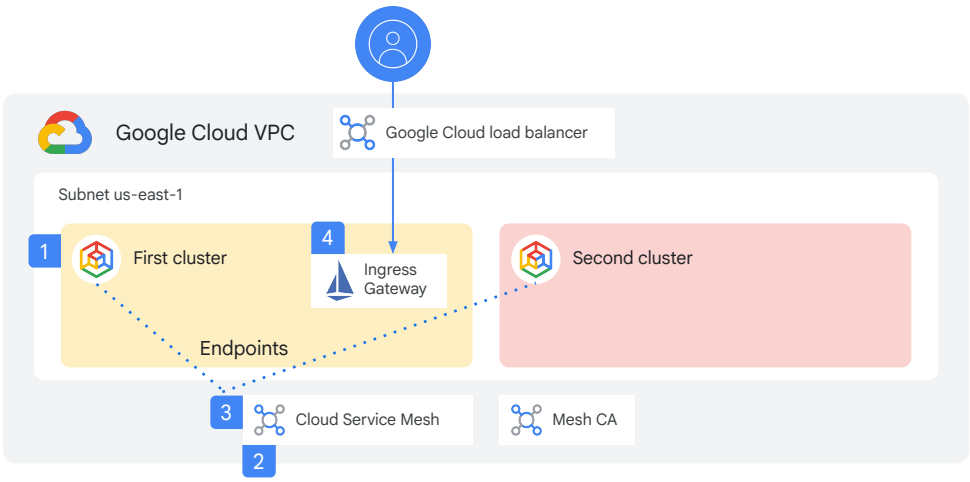
Google Cloud

Next, install Managed Cloud Service Mesh on both clusters. By setting the tag **management: automatic** in the configuration yaml, it will provision Cloud Service Mesh with a managed control plane.

This will handle inter-cluster communication, clusters can access each other's API server for endpoint discovery.

Proprietary + Confidential

Create an ingress gateway to access your workloads



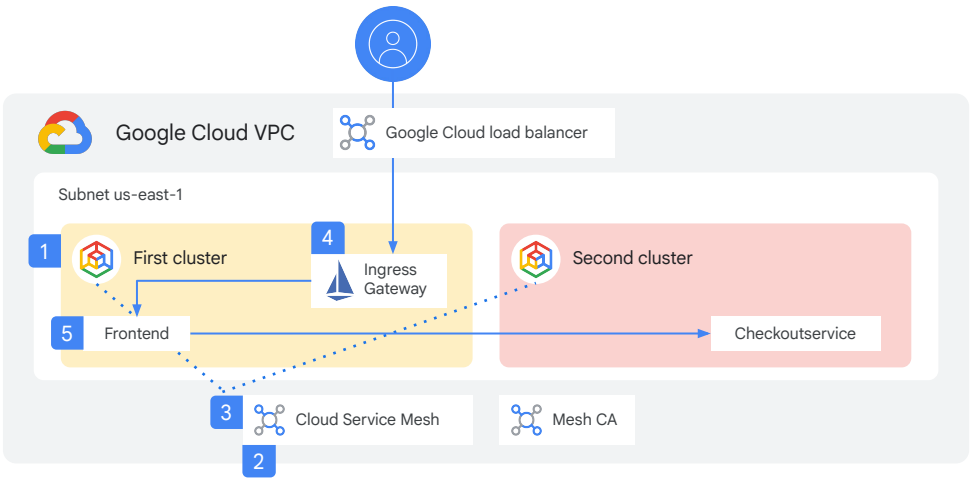
Google Cloud

Optionally, you can use an ingress gateway to access services from outside the mesh. Ingress routing into the cluster is called north-south routing. Ingress gateways can be added to each cluster if desired.

Multi-Cluster Gateways and Multi-cluster Services can be used to balance requests from Google's load balancer to one of the gateways.

Proprietary + Confidential

Deploy your application across clusters



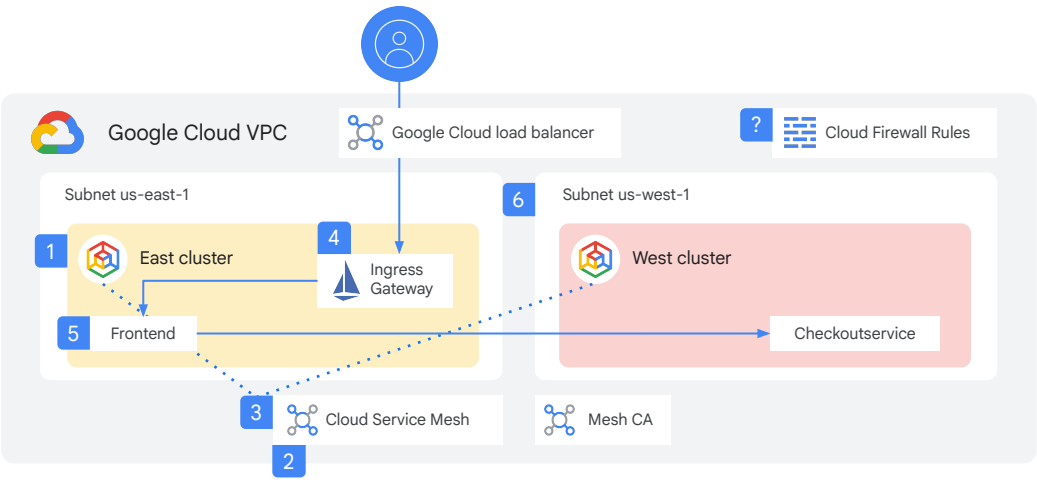
Google Cloud

Next, label namespaces for automatic sidecar injection, and deploy your application across clusters.

Services in one cluster will automatically be able to communicate with services deployed on the other cluster.

Proprietary + Confidential

Variant 1: Multi-region clusters

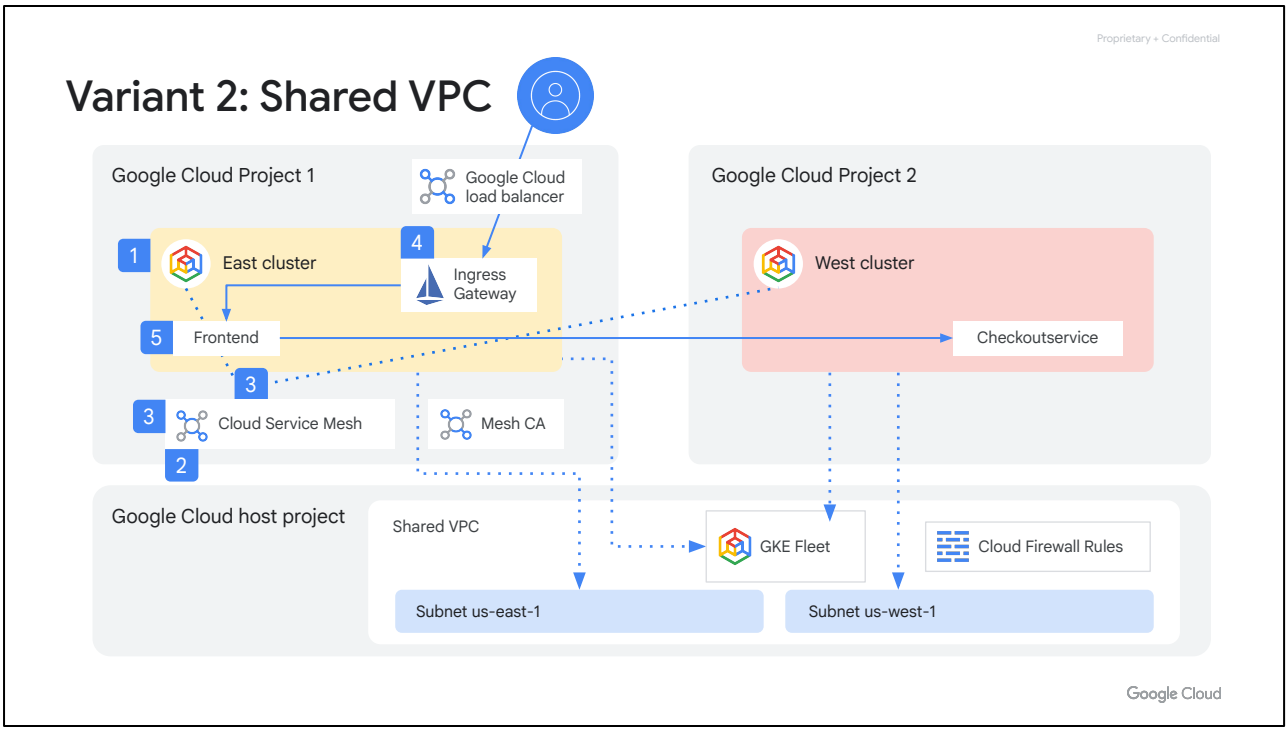


Google Cloud

This configuration has several variants.

You may have clusters located in multiple regions with different subnets. In this scenario, you must explicitly set up firewall rules to allow cross-subnet traffic.

The firewall rules must include all CIDR ranges in the cluster, including nodes, services, and pods.

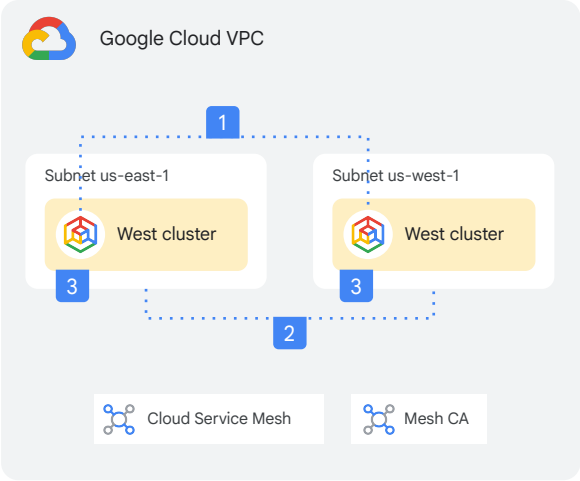


Another variant is to use a Shared VPC. Network configuration is centralized in a host project. A Shared VPC lets service teams in other projects use the network for their own infrastructure.

A centralized networking or security team must create firewall rules on the host project that enable communication between all CIDR ranges in both clusters. Ensure that clusters are not registered to the same fleet in the host project.

Proprietary + Confidential

Variant 3: Private clusters in different subnets



Google Cloud

You can also use private clusters, which limit Kubernetes API server accessibility to services on the same VPC network.

We are going to use managed Cloud Service Mesh in this example. If you wish to use a in-cluster Cloud Service Mesh you must configure with private IPs, as public IPs are not accessible.

There are three steps to enable communication between private clusters.

First, set up endpoint discovery by manually creating secrets. These secrets should contain the private Kubernetes API server IP address of each cluster, and they need to be created in the other cluster.

After endpoint discovery is set up, add the pod address range of the calling cluster to the authorized networks of the remote clusters. This allows the service mesh control plane in each cluster to call the GKE control plane of the remote clusters.

Finally, enable global control plane access to allow the service mesh control plane in each cluster to call the GKE control plane of the remote clusters.

However, this brings some challenges because clusters are no longer able to communicate with each other. To make that communication possible, we need to perform three actions.

First, configure endpoint discovery, we must get the private Kubernetes API server IP from each cluster and create a secret pointing to that IP in the other cluster. That way,

clusters are able to read the pod endpoints registered in the remote cluster.

Second, the the Cloud Service Mesh control plane in each cluster needs to call the GKE control plane of the remote clusters. To allow traffic, you need to add the pod address range in the calling cluster to the authorized networks of the remote clusters.

Third, enable control plane global access to allow Cloud Service Mesh control plane in each cluster to call the GKE control plane of the remote clusters.

Proprietary + Confidential

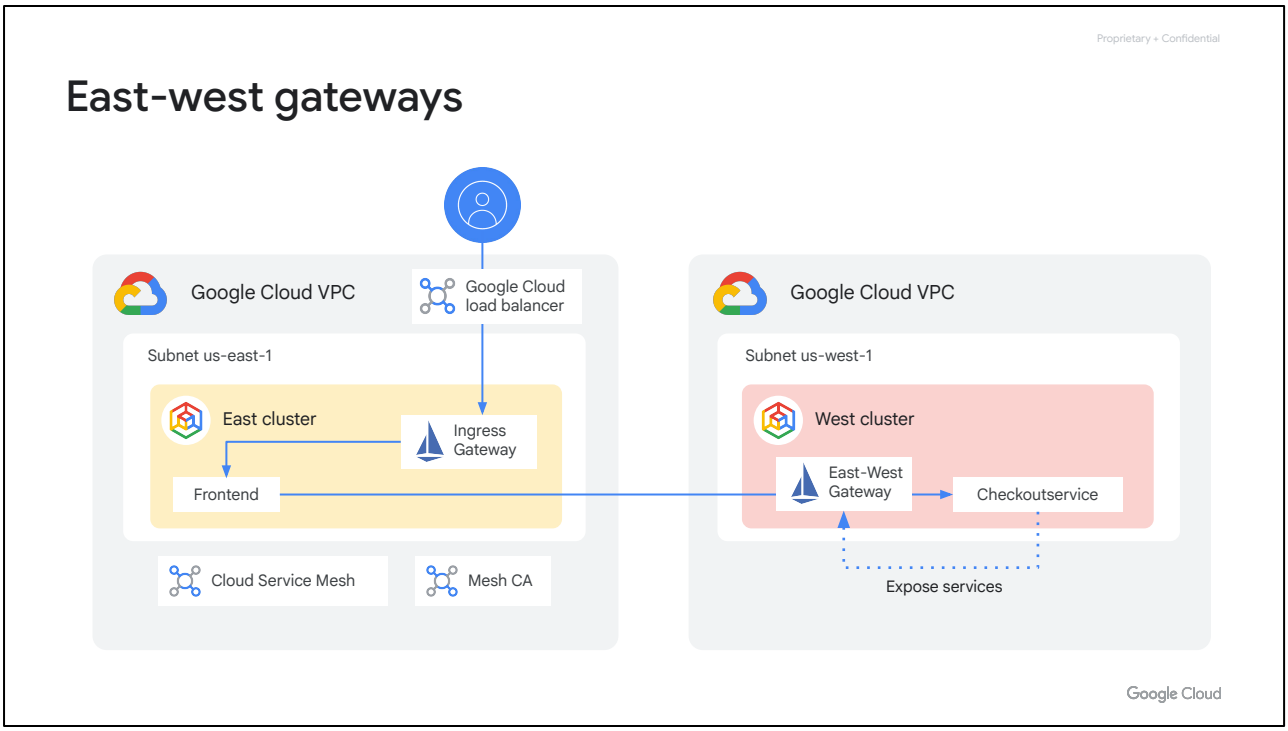
Multi-cluster Networking with Cloud Service Mesh

01 Single network east-west routing

02 Multiple network east-west routing

Google Cloud

In addition to running multiple clusters on the same network, another common configuration is running multiple clusters on different networks. When deploying clusters in multi-cloud or hybrid environments, you might need to enable communication between them without relying on VPNs. Let's explore how to achieve this.

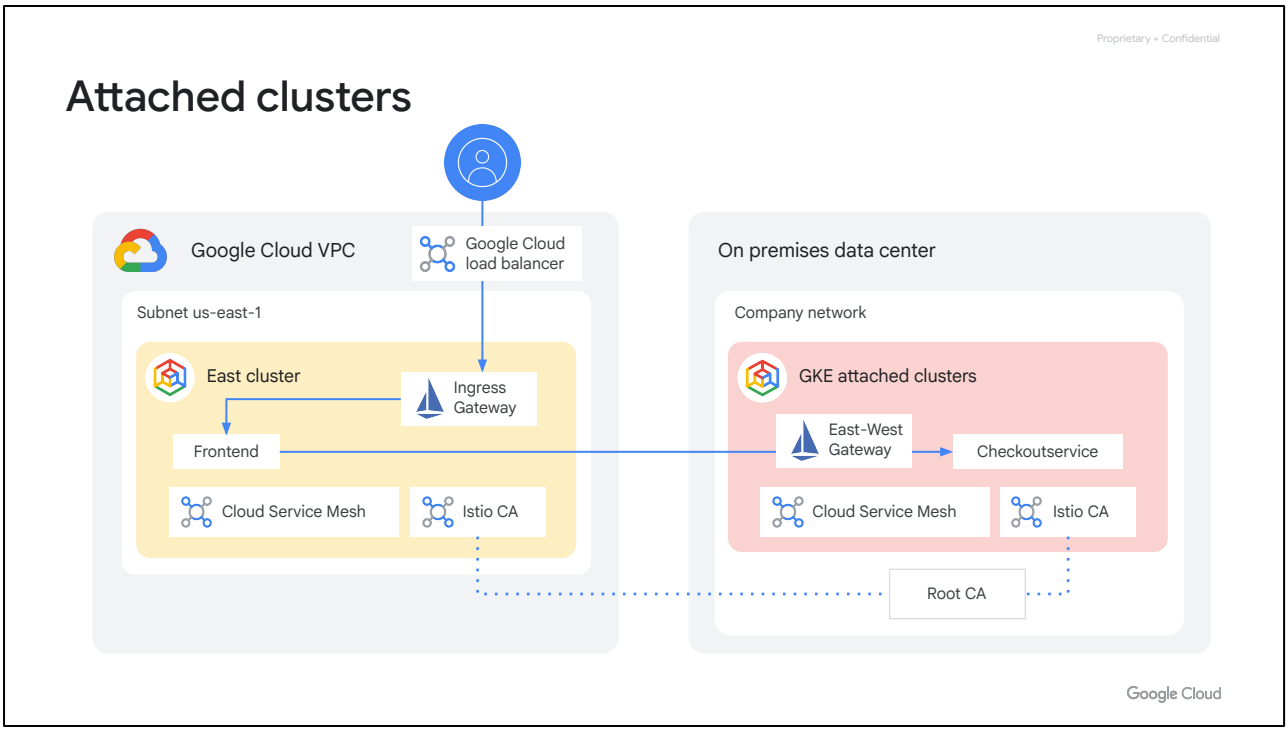


When VPC peering or Cloud Interconnect are not feasible for connecting GKE clusters across different VPC networks, an East-West Gateway can be used to facilitate communication.

This gateway acts as a proxy, enabling controlled communication between services in different clusters. It's important to understand that the East-West Gateway is designed to manage inter-cluster traffic, particularly in scenarios where direct VPC peering or Interconnect is not an option. For each service intended to be accessible across clusters, you configure the gateway accordingly. This allows for controlled visibility into cross-cluster service communication.

Although the gateway is publicly accessible, services can only be accessed by services with a trusted mTLS certificate and workload ID.

Production systems may require additional access restrictions like firewall rules to prevent external attacks.



In this example, a mesh spans across a GKE cluster and an attached cluster running in a customer's location.

Attached clusters are GKE-compatible, not managed by Google, and must be registered to GKE.

Examples of attached clusters include Amazon EKS, Microsoft AKS, Red Hat Openshift (both OKE and OCP), KIND, K3s, and K3d.

Support for these clusters varies, and you might need to consider the option of using Open Source Istio depending on your needs.

To establish trust between multiple clusters, use Root CA for certificate authority.